

APAI LAB 06: On-Device Learning

Călin Diaconu – calin.diaconu2@unibo.it
Ahmad Mirsalari – seyedahmad.mirsalar2@unibo.it
Ihor Sachok – ihor.sachok@unibo.it



01. Lab Information

01. Lab Information – Objective of the Class

- **Subjects**:
  - Understand the training loop
  - Understand particularities of on-device learning
- **Programming language**: C
- **Lab duration**: 3h

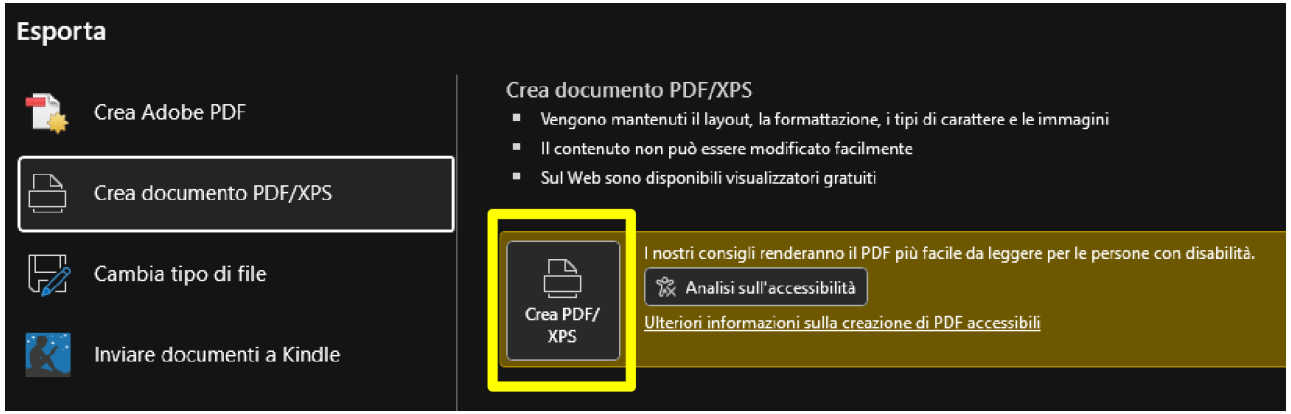
- **AI policy**: we will not prevent you from using AI in solving the assignments, but the lab is meant to be a hands-on learning experience. To keep it useful, we have included an .md file (also used in a Stanford class), that should be automatically read by AI agents, in order to prevent them from giving you a direct solution and rather guide you towards finding it yourself.

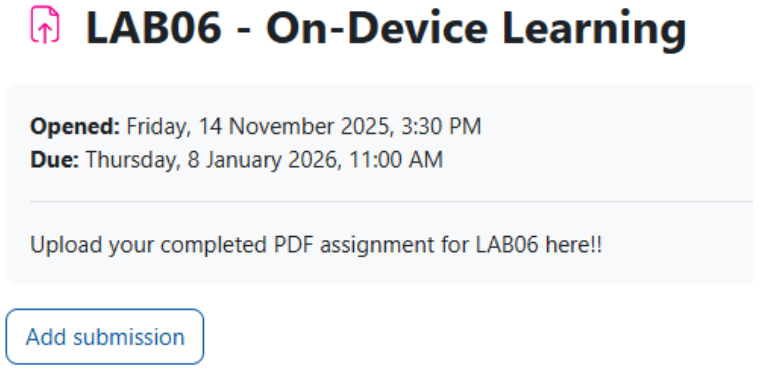
- The class is meant to be interactive: coding together and on your own!

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

01. Lab Information – How to Deliver the Assignment

- You will deliver **ONLY THE PDF** assignment, **NO code**
  - Download the **deliverable template** from Virtuale
  - **Fill in the results** required by the assignment
  - **Export to PDF** format (this option is available in most document editors – Google Docs, Office Word, or as a “Print to PDF” option in the Print menu)
  - **Rename the file** to *LAB<number_of_the_lesson>_APAI_<name_surname>.pdf*
  - Use the **Virtuale platform to upload the PDF** before the **deadline (Sunday 22nd of November 2026, end of day, 23:59)**
  - For each lab that you solve and submit successfully, you will receive **2 extra points (10 maximum in total, for mod. 2 labs)**.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



02. Prerequisites

02. Prerequisites – Containers & Docker



STEPS ONLY FOR PERSONAL MACHINES

- **What is a Container?**
  - A lightweight, portable, and self-contained environment that bundles together
    - A consistent runtime environment
    - Your application code
    - All its dependencies (libraries, tools, configurations)
- **What is Docker?**
  - The most used platform for working with containers, which allows the user to:
    - **Build images**, defined in a Dockerfile
    - **Run containers** – isolated environments, created according to the "recipe" defined inside an image
    - **Share images with other people** through the Docker Hub, for reuse and consistency in development
- The **Docker image** used in this course includes the PULP-SDK and its related compilation toolchain

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Docker Setup (WSL install)



STEPS ONLY FOR PERSONAL MACHINES

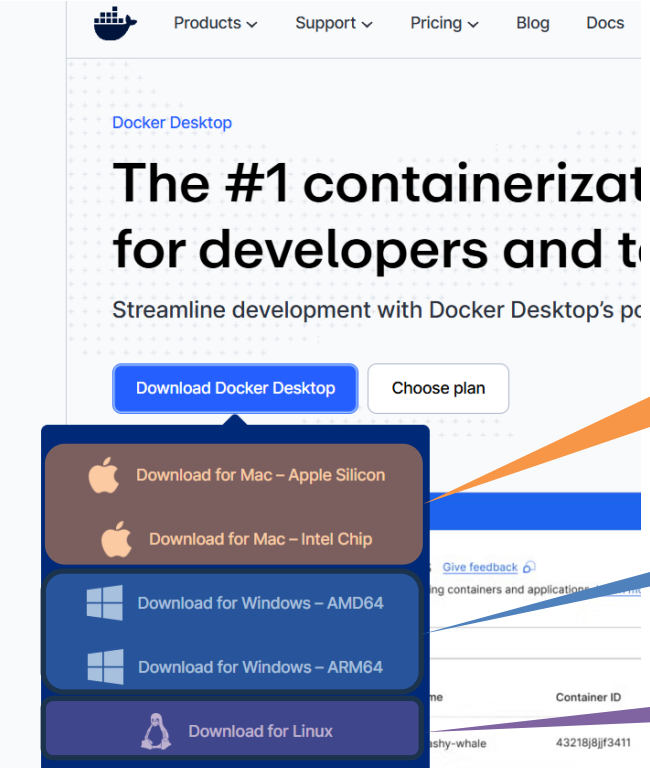
- **Step 0** (for **Windows users only!**) – install Windows Subsystem for Linux (WSL 2)
  - **What is WSL?**
    - **A compatibility layer** that lets you run a genuine Linux environment directly on Windows, without a separate virtual machine or dual boot
    - Gives access to Linux command-line tools and applications, running natively alongside your Windows apps
    - It can run both command-line and graphical Linux apps directly inside Windows
  - The **advantage** of running Docker through WSL is that, while in both cases Docker runs through a virtual machine, the WSL is much lighter and it doesn't slow down the entire system
  - If you plan to work on the lab on a **Windows** machine, please **follow these installation instructions for WSL**

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Docker Setup

STEPS ONLY FOR PERSONAL MACHINES

- **Step 1 – install Docker Desktop**
  - Choose, download and install the proper version from the official website.



- Apple Mac versions
- Choose based on your computer's processor

- Windows versions (AMD64 for most, unless you are **VERY SURE** that your processor is ARM64)
- **CHOOSE THIS EVEN IF YOU WORK WITH WSL!**

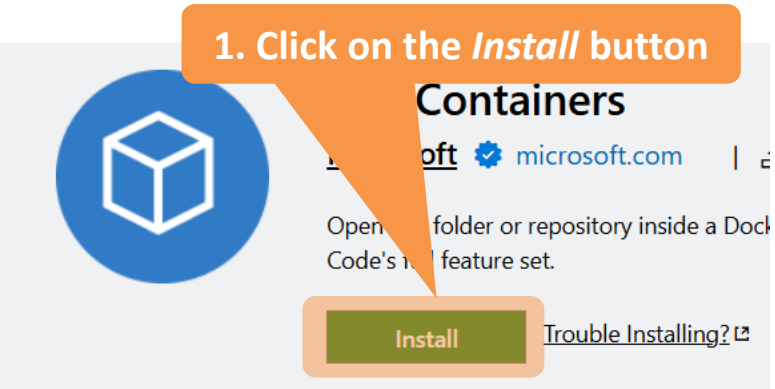
- Universal Linux version
- **If working with WSL, choose a Windows version!**

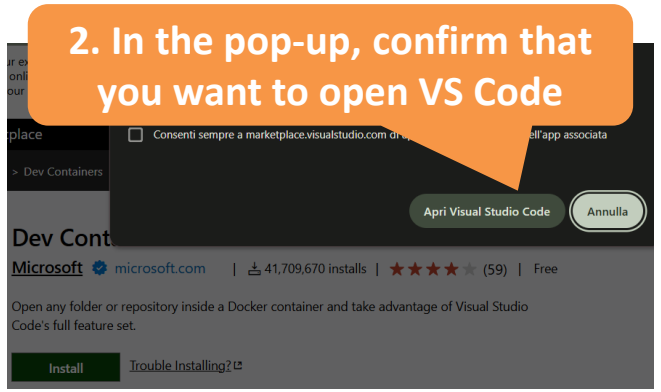
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

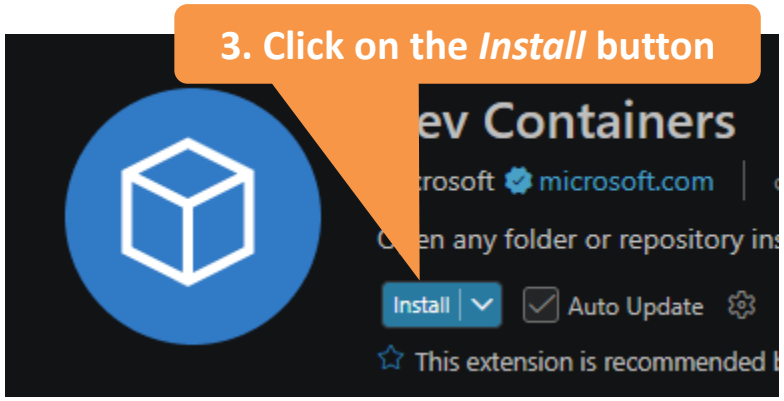
02. Prerequisites – VS Code

STEPS ONLY FOR PERSONAL MACHINES

- **Step 2 – Install Visual Studio (VS) Code**
  - **What is VS Code?**
    - A free, popular and versatile text editor from Microsoft
    - It is highly customizable through a large extension library
    - The one that interests us is the **Dev Containers** extension, that enables VS Code to work with Containers
  - **How to install?**
    - Choose, download and install for your operating system from the official website
    - **Once VS Code is installed**, we need to install the extension for working with Containers
      - Find and install it from the VS Code Marketplace







ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Error Solving

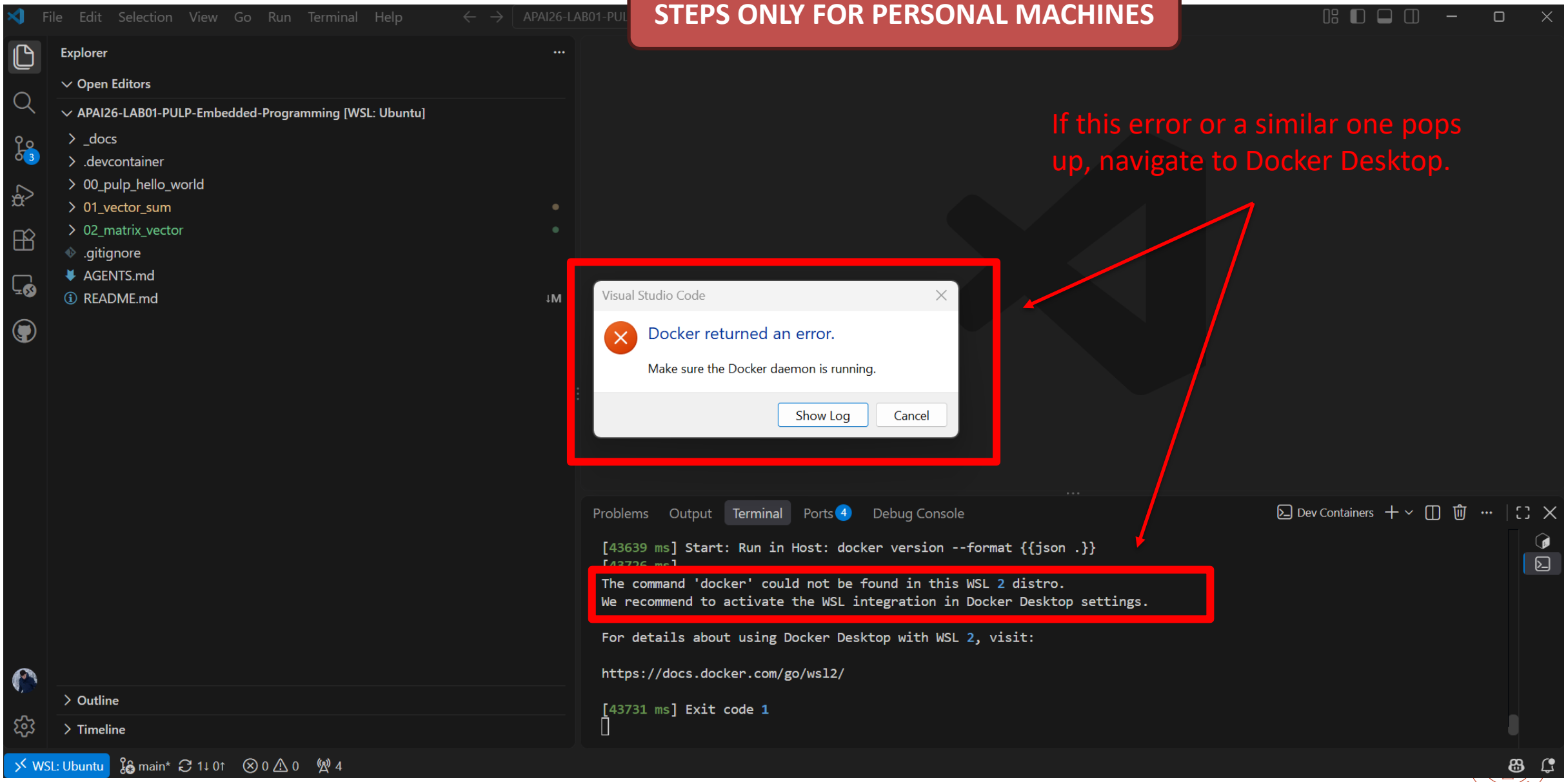
STEPS ONLY FOR PERSONAL MACHINES

IN VS CODE, IF DOCKER RETURNS AN ERROR,

FOLLOW THE INSTRUCTIONS IN THE FOLLOWING SLIDES

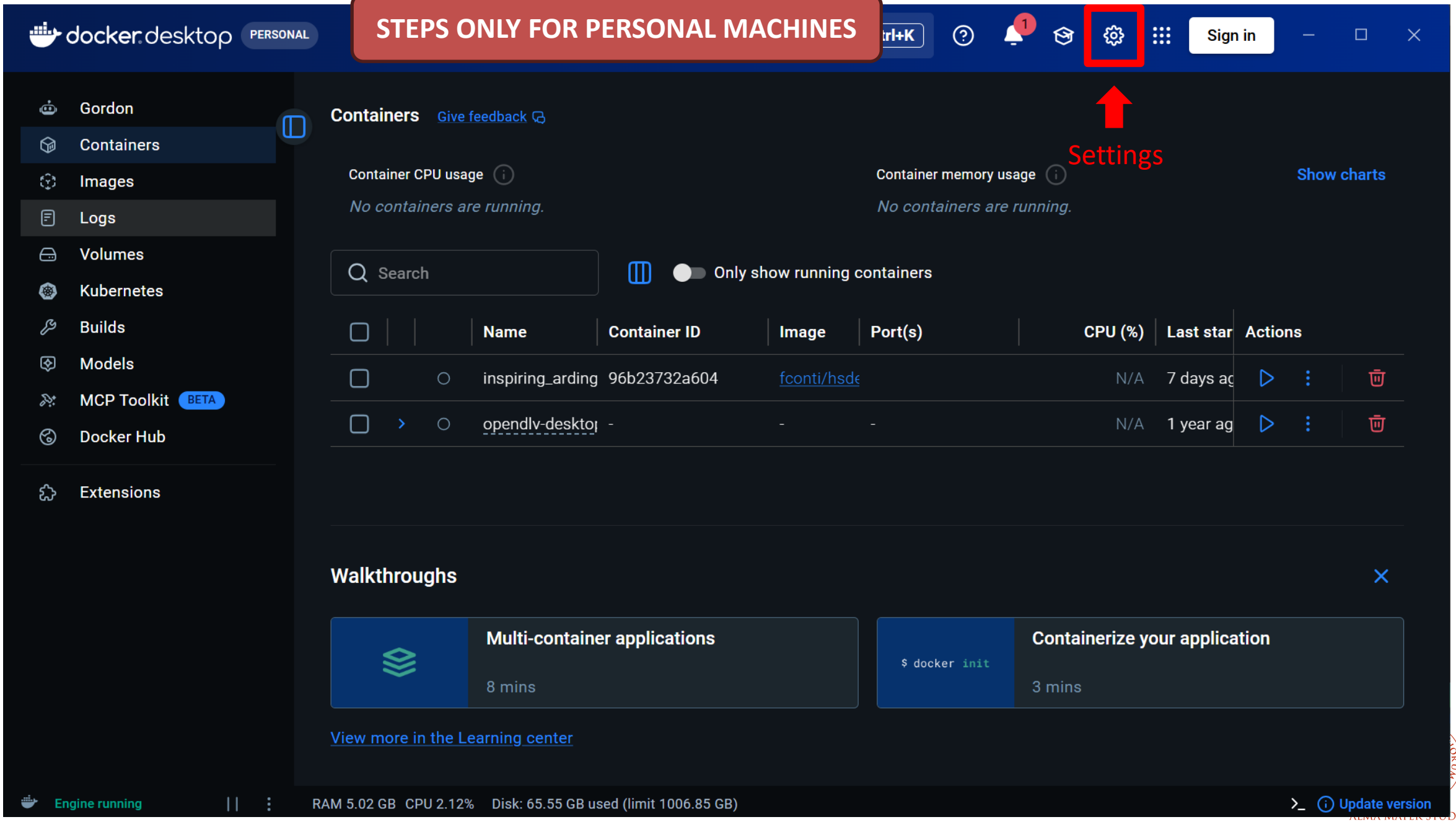
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

STEPS ONLY FOR PERSONAL MACHINES

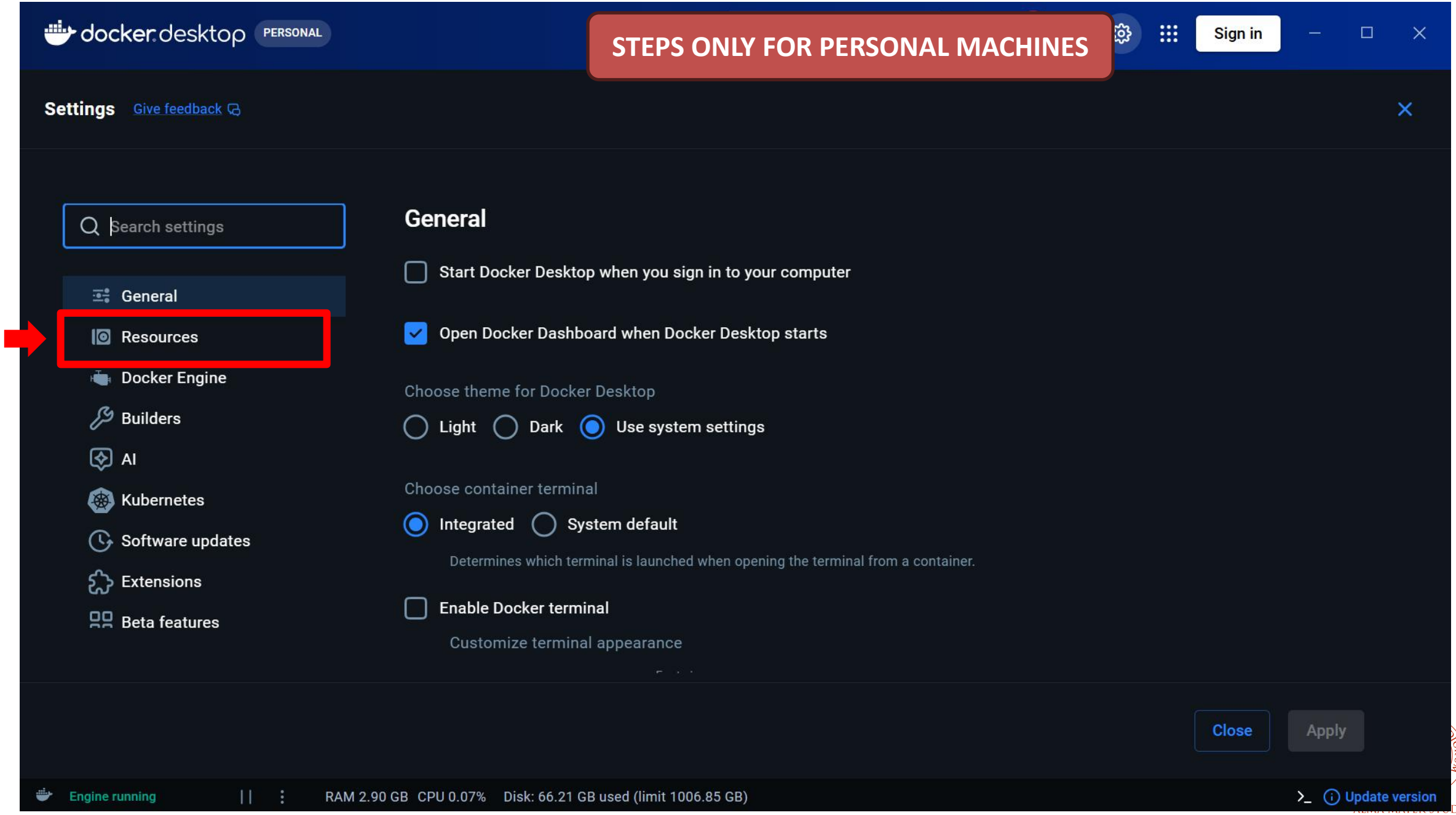


If this error or a similar one pops up, navigate to Docker Desktop.

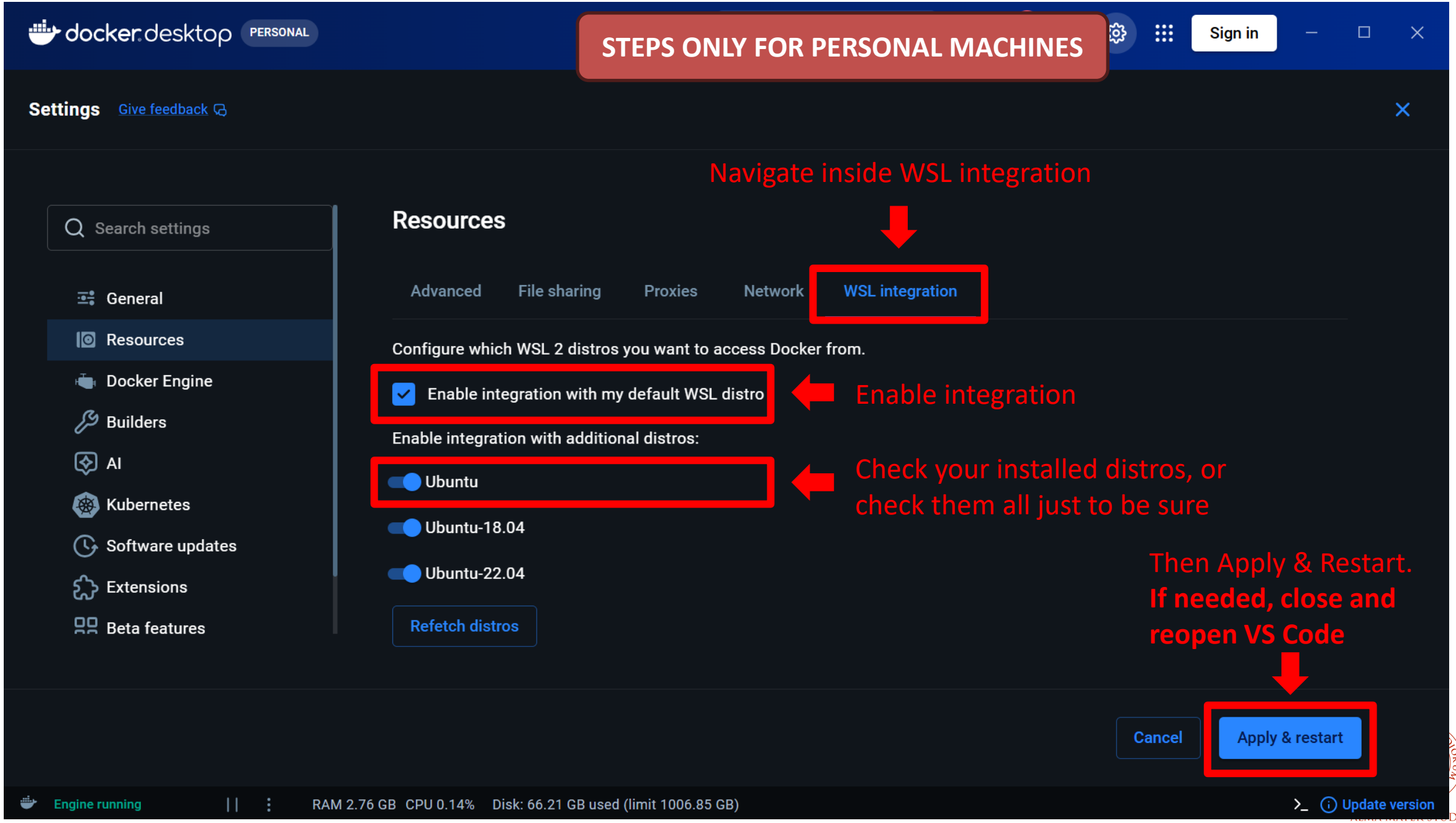
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

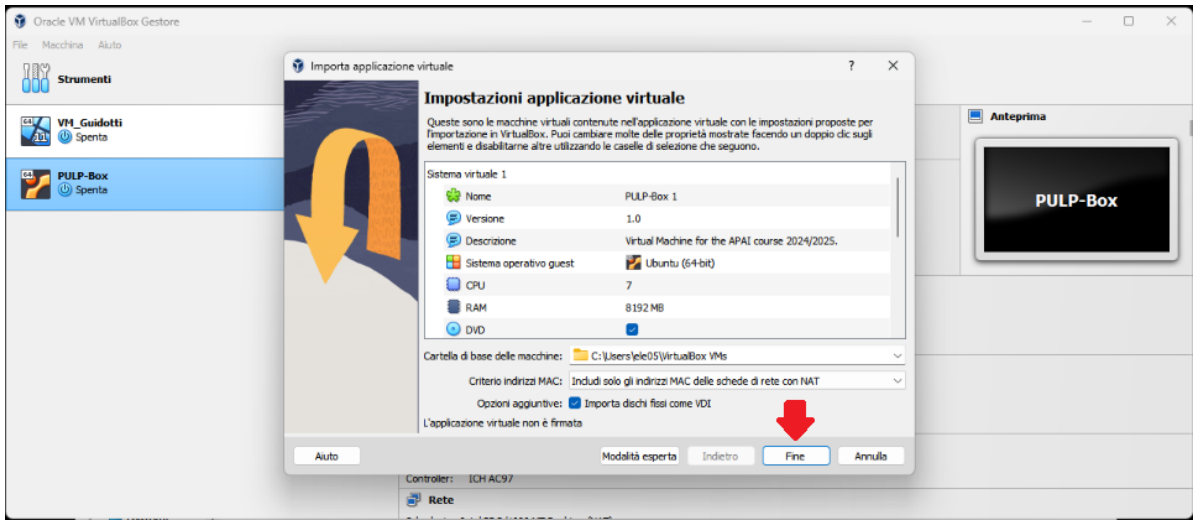
02. Prerequisites – Virtual Machine – Part 1

STEPS ONLY FOR LAB MACHINES

- When using the **lab computers**, you will find an Ubuntu-based virtual machine that already contains everything that is needed for solving the assignments:
  - **PULP-SDK –** so we don't need the Docker container anymore
  - **VS-Code –** in this case, only used for code editing and accessing the terminal
- You will only be required to **download the resources** for the lab, which are also **on Virtuale** (the only website accessible from the lab computers) as a **zip file**. This must be done from a browser **OUTSIDE of the virtual machine**. This is also a good step to **unzip** it, with a simple *right-click -> Extract*

Once you have this zip file, follow these steps:

- **Step 1**: Go to *C:/VM_APAI* and double-click *PULP-box.ova*. When VirtualBox opens, click on *"Finish"/"Fine"*

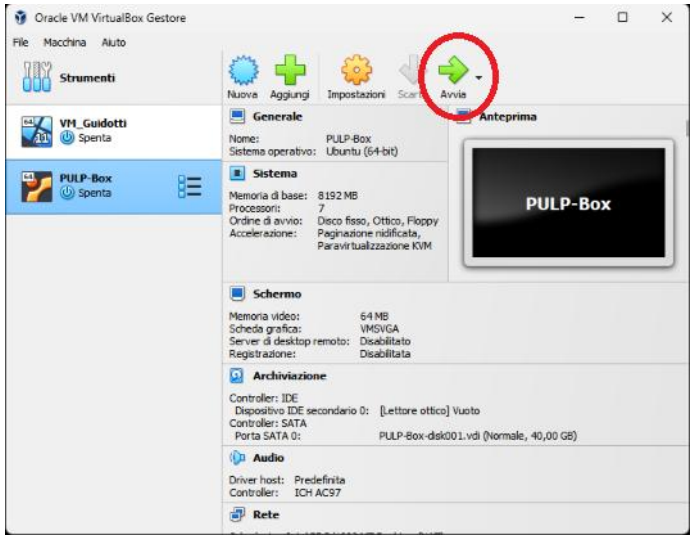


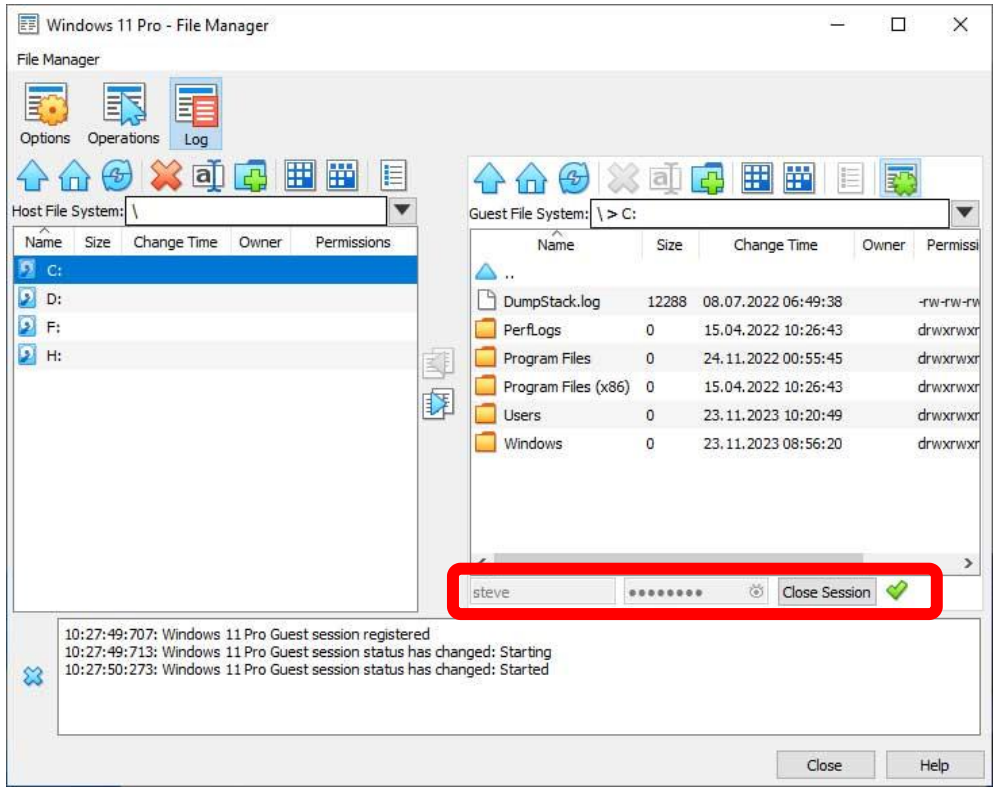
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Virtual Machine – Part 2

STEPS ONLY FOR LAB MACHINES

- **Step 2**: Once the VM is imported, open it by clicking on *"Start"/"Avvia"*
- **Step 3**: For login – the username is ***pulp-box*** and password is ***pulp***
- **Step 4**: Access the files of the VM:
  - From the **top-bar menu**, access ***Machine -> File Manager***
  - In the **left** panel, access files from the **PC**, in the **right**, access files from the **VM**
  - In the **bottom of the right panel**, you will have to **enter the login credentials**





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Virtual Machine – Part 3

STEPS ONLY FOR LAB MACHINES

- **Step 5**: With a simple drag-and-drop, **move the files from the PC location into a VM** location you choose (it must be a subdirectory of the home directory to have file writing rights)
- **Step 6**: Inside the VM, open a terminal and set up the PULP SDK by running

```
source setup_pulp_sdk.sh
```

- **Step 7**: Go to the VM location you chose to put the assignment data in, using the *cd* command in the terminal

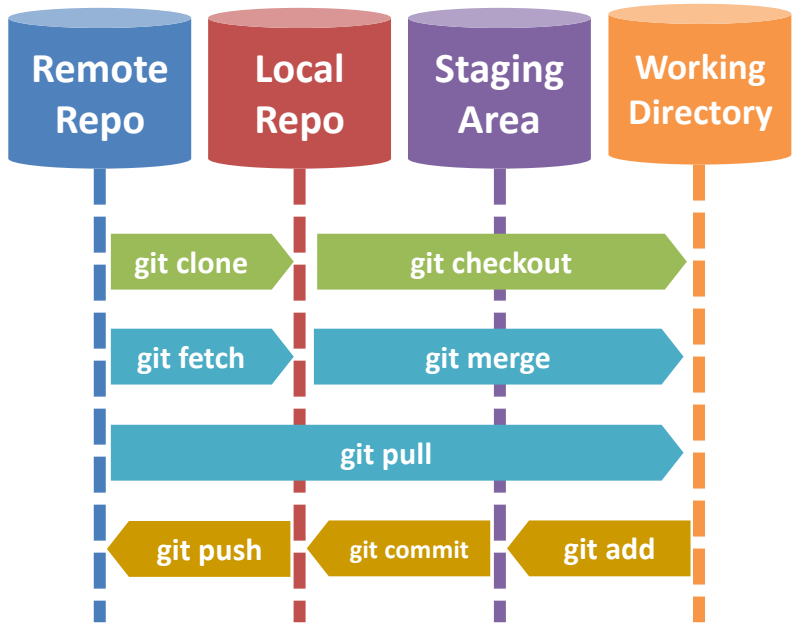
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – git

Link to a useful intro git tutorial



- **What is Git?**
  - A free, open-source version control system that tracks changes to files over time.
  - It lets multiple people work on the same codebase (repository) and merge changes (commits) without overwriting each other's work.
  - It keeps a full history of changes, so you can review, compare, or revert to any earlier version.
- **Common commands we will use**
  - **git clone –** Downloads a full copy of a remote repository to your machine
  - **git checkout** – Switches your working directory to a different branch or commit
  - **git fetch –** Checks the remote for new changes without applying them locally
  - **git merge –** Combines changes from one branch into another
  - **git pull –** Fetches (*downloads)* remote changes and merges them into your local branch (equivalent to *git fetch* + *git merge*)
  - **git add [file1, file2, …] –** Stages (*marks*) the specified files to be included in the next commit
  - **git commit –m "commit message" –** Saves the staged changes to your local repository's history with a message
  - **git push –** Uploads your committed changes to the remote repository



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

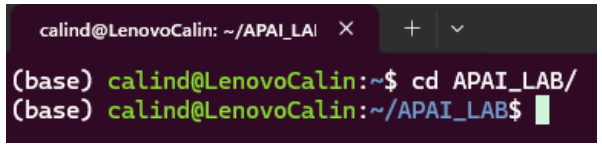
02. Prerequisites – Hello, World Example – Part 1

STEPS ONLY FOR PERSONAL MACHINES

In order to confirm that everything was installed correctly, we will now run the ***Hello, World*** basic example available in the repository of this lab.

The **example is run on WSL, on a Windows** machine, but it should work identically on other operating systems.

- **Step 1:** In a terminal, open the location where you wish to work



- **Step 2:** Clone the repository of lab 5 locally

```
git clone https://github.com/EEESlab/APAI26-LAB06-TinyTransformers
```

- **Step 3:** Enter the cloned directory

```
cd APAI26-LAB06-TinyTransformers/
```

- **Step 4:** Open the directory in VS Code

```
code .
```

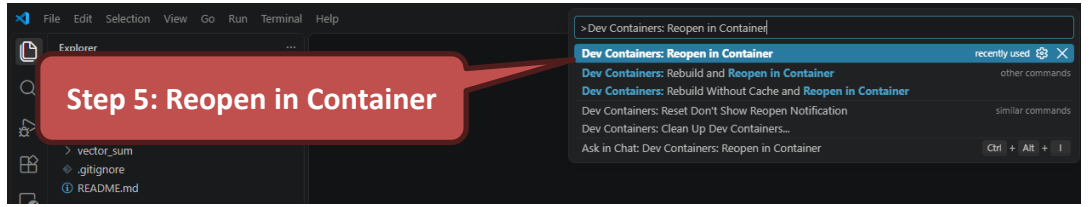

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Prerequisites – Hello, World Example – Part 2

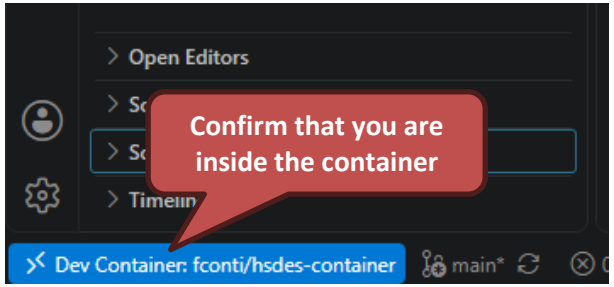
STEPS ONLY FOR PERSONAL MACHINES

* From this point on, we will only work from VS Code, so it is ok to close the terminal you have previously opened.

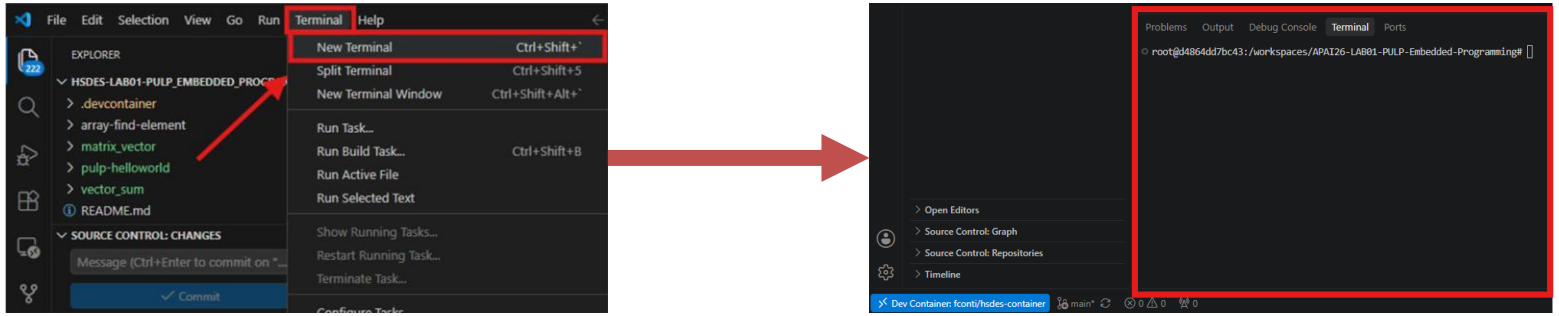
- **Step 5:** At this point, the project is open in the local *environment*. We need to **reopen inside the Docker container**.
  - **Step 5.1**: Press **CTRL** + **Shift** + **P**
  - **Step 5.2**: Type in ***"Dev Containers: Reopen in Container"***
  - **Step 5.3**: Press **Enter**



* We are now **inside the container**. From now on, when opening VS Code, the project will **automatically open inside the Docker Container**. It is confirmed in the bottom-left corner of VS Code.



- **Step 6:** Open a **terminal inside VS Code** (which will be **inside the container**)
  - Access the terminal through the **top-left task bar (*Terminal -> New terminal*)**
  - The terminal **will open in the bottom** of the window.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



03. Introductory Information

03. Introductory Information – Forward Step



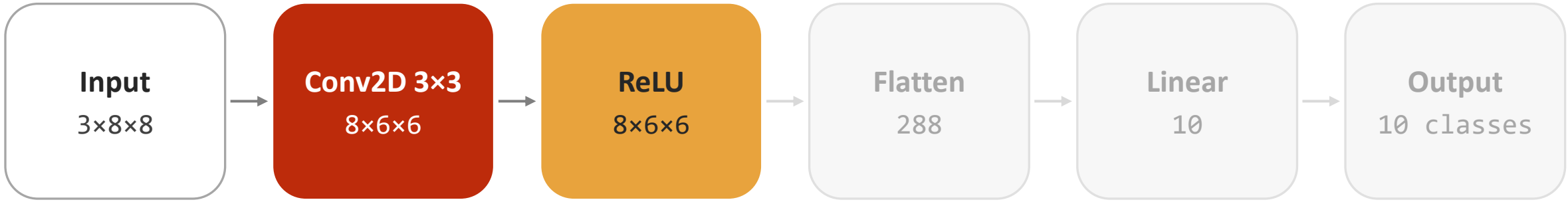
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Forward Step



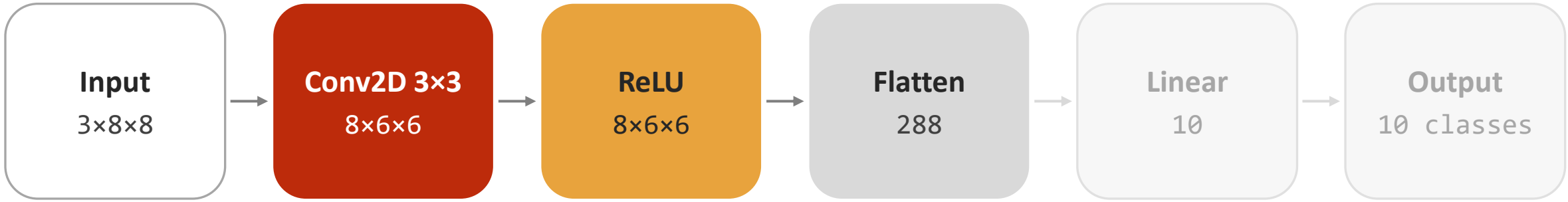
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Forward Step



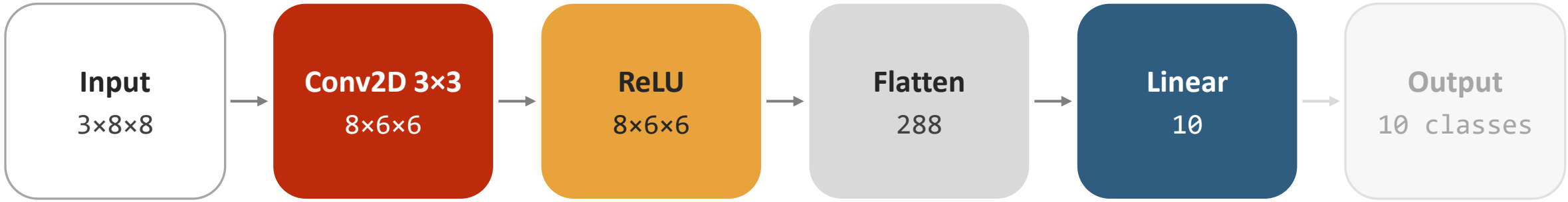
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Forward Step



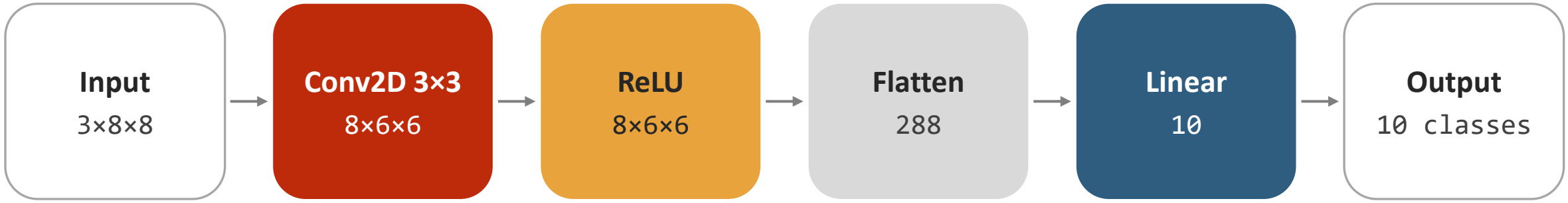
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Forward Step



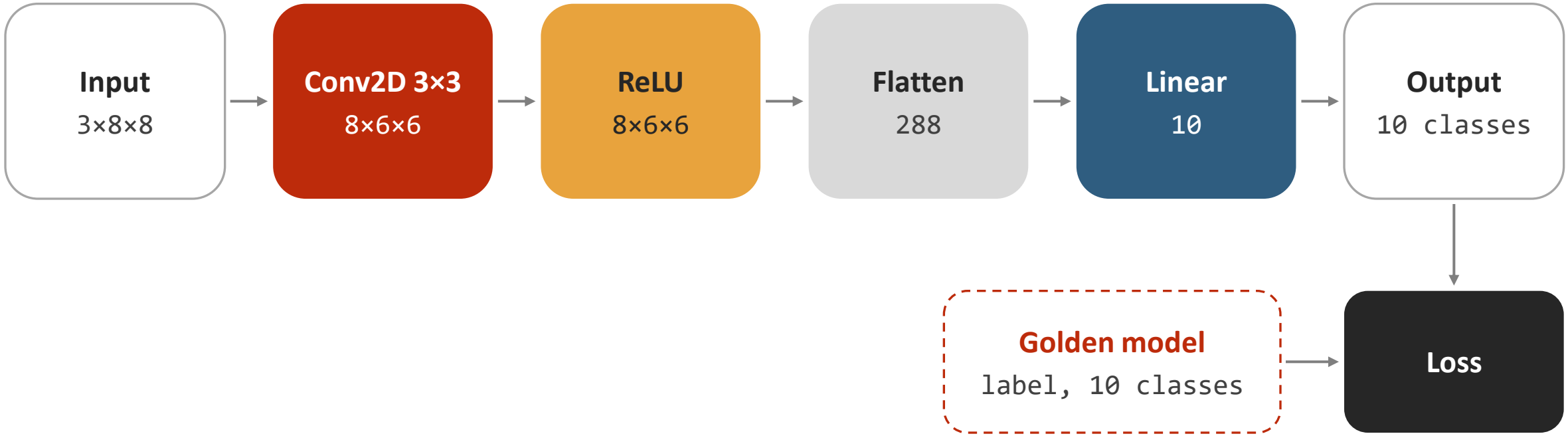
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Forward Step



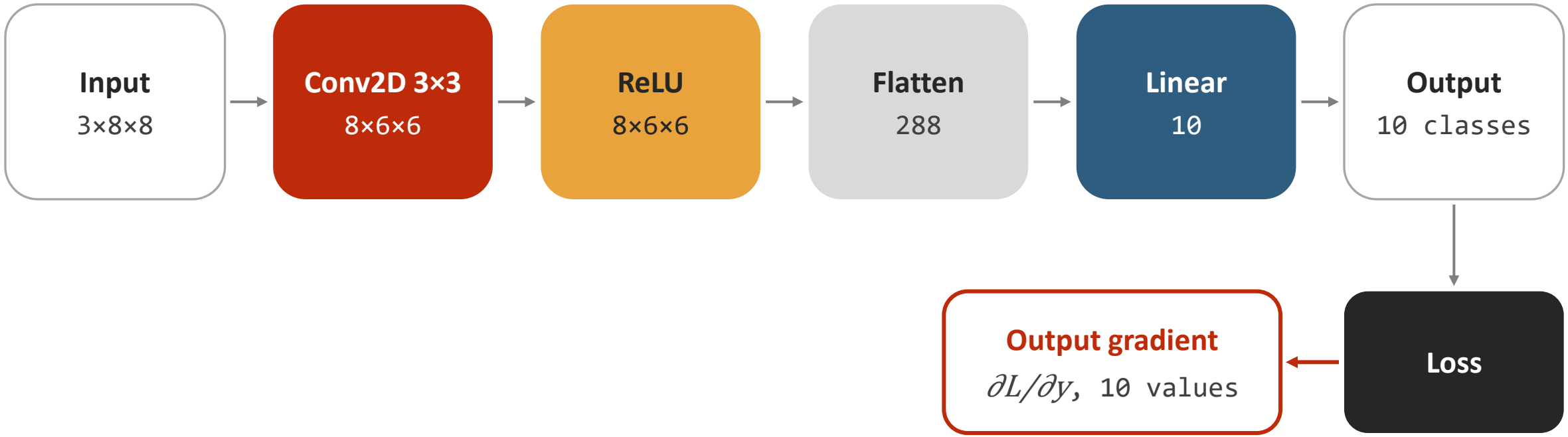
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Compute Loss



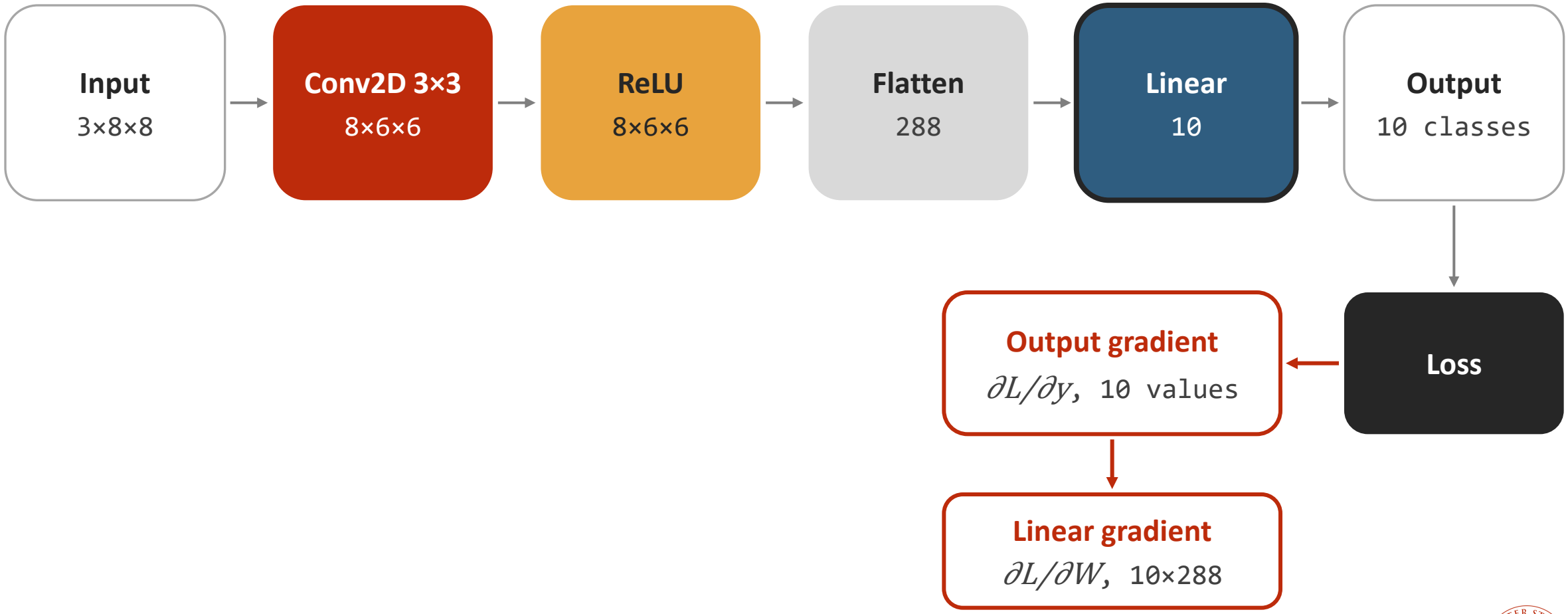
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Compute Output Gradient



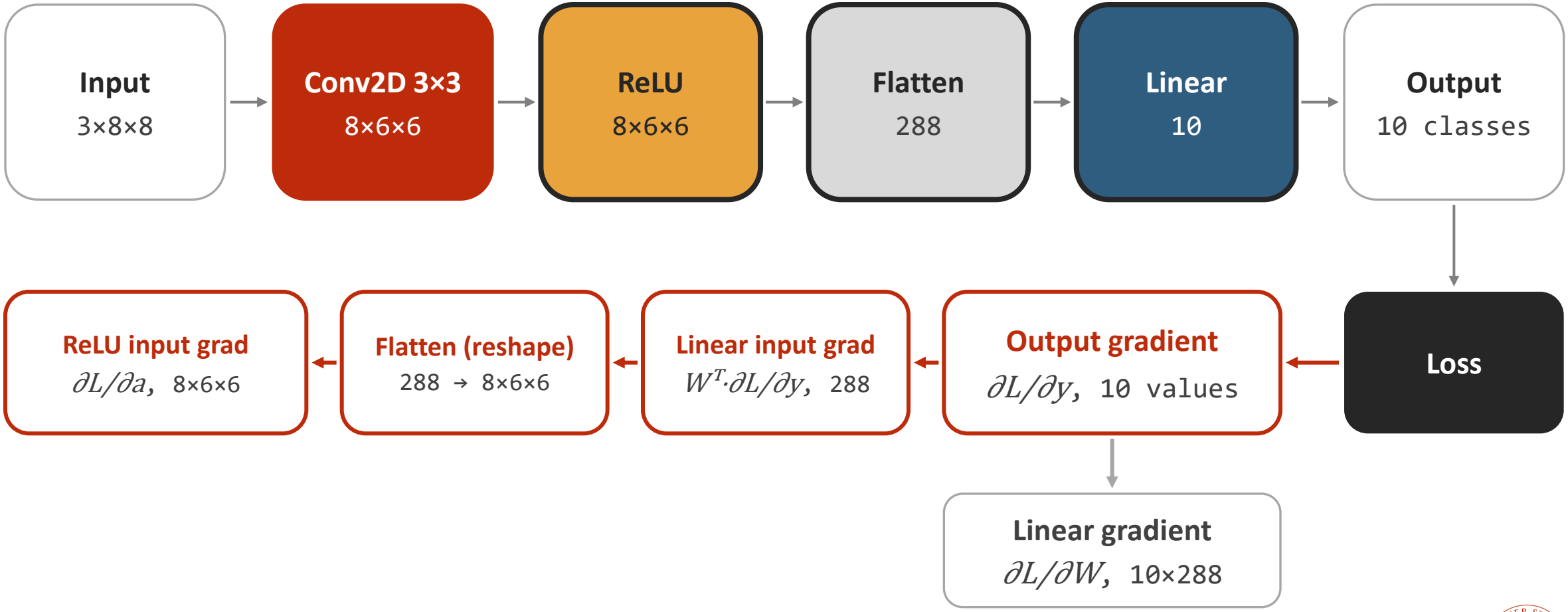
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Compute Linear Weight Gradient



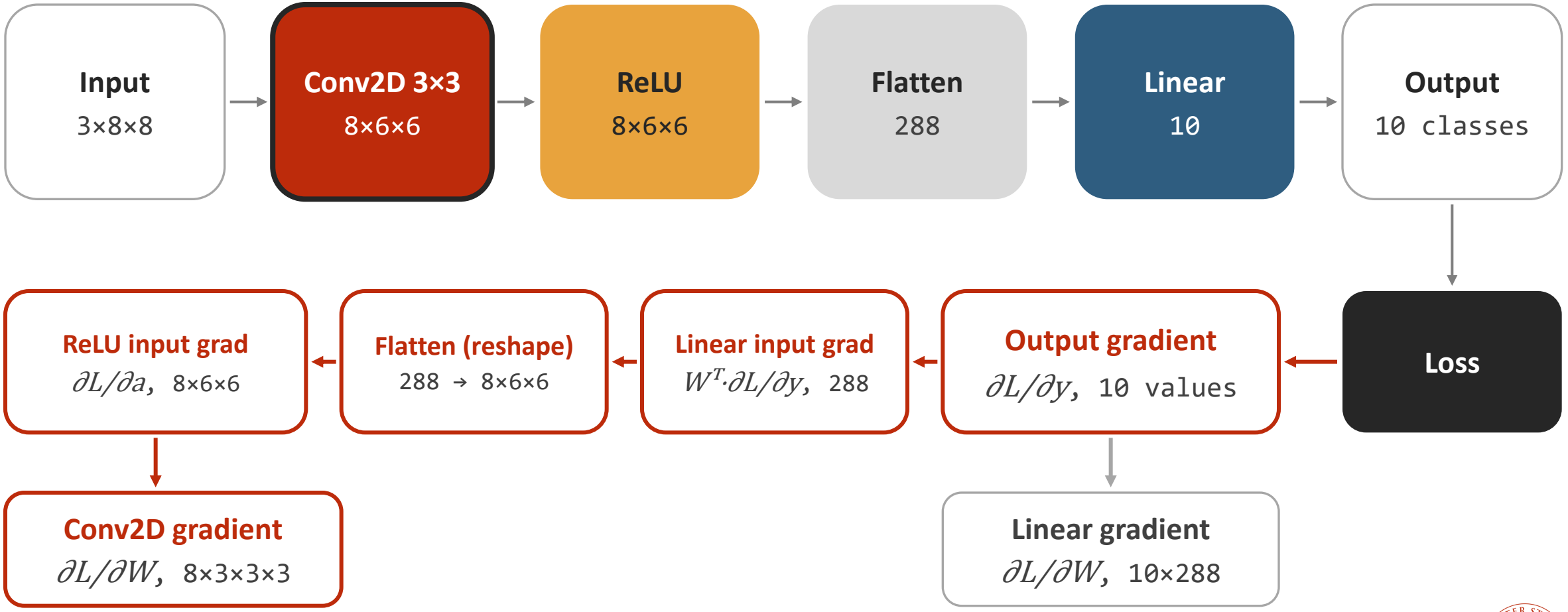
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Compute Inputs Gradients



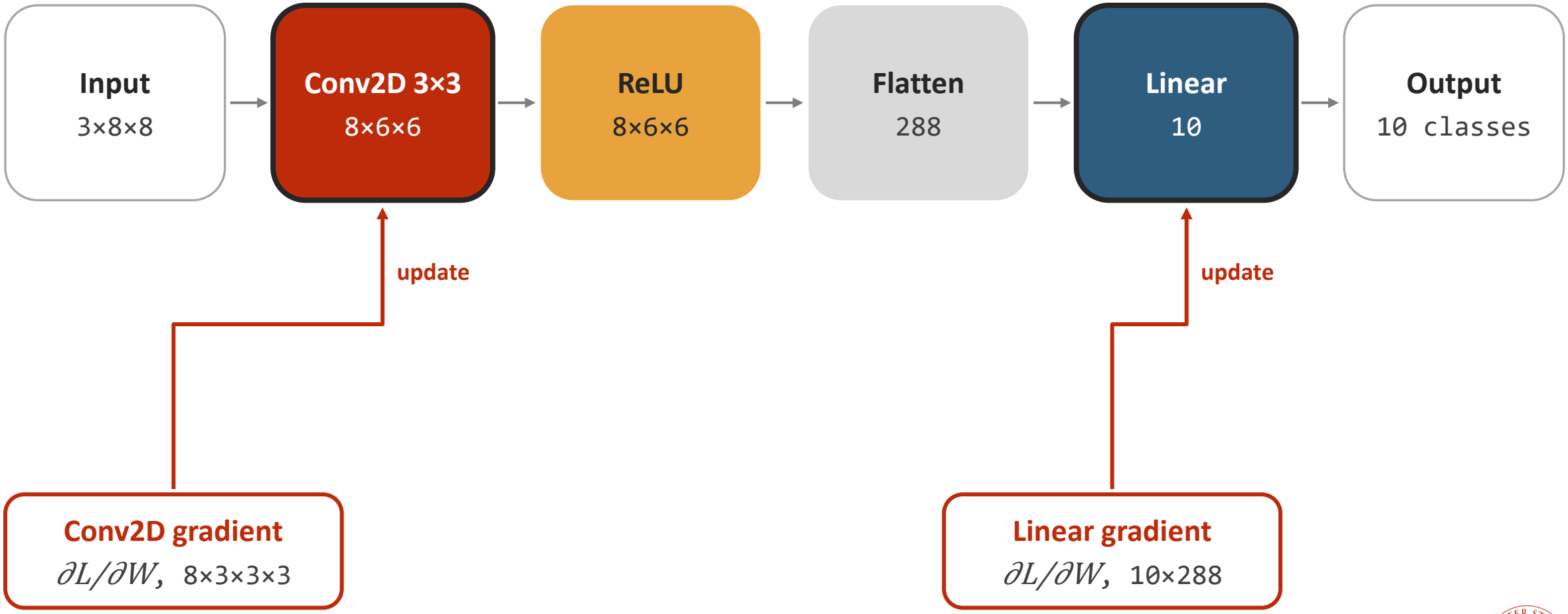
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Compute Conv2D Weight Gradient



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

03. Introductory Information – Update Weights



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



04. Assignment

04. Assignments – Task 1 – Analyze Project

- Since we work on a demo project, the data involved in model computations is **automatically generated** using a Python library. It's used to obtain a so-called "**Golden Model**" – the reference that's considered to be mathematically correct, against which the values obtained by the DNNs will be compared.
- These scripts are in `utils/GM.py`.
- After the first run of the project, you can find the generated data in `io_data.h`. Run this command in the terminal (which will be required for the other tasks as well):

> **`utils/GM.py`**
> Golden Model scripts (Python)

> **`io_data.h`**
> Generated data, after the first run

```
make clean get_golden all run
```

- You will now be able to answer **question 1A**.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 1 – Forward Step

Forward step

- We now need to perform a forward prediction with the **non-initialized** DNN.
- In `net.c`, check the `forward()` function – it performs a forward prediction by calling all the layers in sequence.
- Each layer **computes its output** using the data provided as **output by the previous layer**.
- The **first layer takes the input data** as input, and the output of the **last layer is the DNN's prediction**.
- We will now check what happens during the forward step.
  - First, modify the `net_step()` function to perform a forward prediction.
  - Then, print the output of the DNN and answer **question 1B**.

TIP: The code sections which **require editing** are **marked** as shown in the box to the right (there might be **multiple** of them for a single exercise). Find them by **searching across the whole project** with **Ctrl+Shift+F** inside VS Code.

Performance

- For **questions 1C to 1E,** you need to evaluate the performance of the DNN.
- Like in previous labs, we will use **PULP Performance Counters**. They are managed by the macros contained in `stats.h`.
- To start measuring the stats, call `START_STATS()`. To stop and print the stats, call `STOP_STATS()`.

NOTE: Be careful **not to profile functions/workloads which contain** `printf()` functions!

```
/**
 * EXERCISE XX
 */
CODE
/**
 * END OF EXERCISE XX
 */
```


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 2 – Back-Propagation

- Now, it's time to work on the **back-propagation**.
- First, we need to compute the **loss function** to compute the **prediction error** of our forward function with respect to the ground-truth label.
- Your task is to complete the code of a **mean squared error loss**, which is defined as:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

- Find it in `lib/sources/pulp_losses_fp32.c`, in the function `pulp_MSELoss()`.
- When you're done, you can **visualize the value of the loss** when running the **make** command by uncommenting this flag inside the `Makefile`:

```
APP_CFLAGS += -DDEBUG_LOSS
```


ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 2 – Back-Propagation

- Now, we need to write the function used to compute the **output gradient of the DNN**. To do so, take its definition as reference:

$$(dL/dy)_i = outDiff[i] = \frac{2}{N}(outData[i] - target[i])$$

- **Complete the code** with the output gradient in **the same function and file as before**. Once that's done, you can **check the values again running the full make command**.

- Finally, remember to **comment** the `APP_CFLAGS += -DDEBUG_LOSS` flag in the `Makefile`, so you won't have `printf()` calls during the training.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 3 – Full Backward Step

- Starting **top-down**: the sequence of operations during the backward step is **reversed with respect to the forward**. That's because the output gradient is brought towards the input, computing across the gradient of the weights.
- The sequence of layers to be back propagated is contained into the `backward()` function in net.c. Looking at the `forward()` function as a reference, **complete the backward() function for the whole network**.
- You can look in the ***lib directory*** to find the forward and backward primitives (e.g., conv2d fp32).



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 3 – Full Backward Step

- In most training backends, the fully-connected training steps (i.e., the forward, weight grad and input grad) are **computed as matrix multiplications**.
- Complete the primitives of the **fully-connected** layer (`pulp_linear_fp32.h`). For this type of layers, the weight and input gradients are computed as follows:

Weight gradient

$$\underset{C_{out}\times C_{in}}{\partial L/\partial W} = \underset{C_{out}\times 1}{outDiff} \cdot \underset{1\times C_{in}}{inData^T} \Rightarrow coeffDiff[o][i] = outDiff[o] \cdot inData[i]$$

Input gradient

$$\underset{C_{in}\times 1}{\partial L/\partial x} = \underset{C_{in}\times C_{out}}{W^T} \cdot \underset{C_{out}\times 1}{outDiff} \Rightarrow inDiff[i] = \sum_o coeffData[o][i] \cdot outDiff[o]$$

- W (coeffData) is a $C_{out} \times C_{in}$ matrix, so **coeffDiff** has the same shape as W. **inDiff** becomes the outDiff of the previous layer.
- When you're done, **compile the code again**. Now the gradients should be computed for the entire DNN.
- Now, profile the `backward()` with the performance counters.
- Remember to **remove/comment** the profiling functions around the backward to continue.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 4 – Weight Update

- Now that we have computed the weight gradient, you can now **use it to update the weights** and improve the DNN.
- It is done through the `update_weights()` function. It calls the optimizer, here Stochastic Gradient Descent, to update the weights with the formula:

SGD update

$$W[i] = W[i] - LR \cdot \frac{dL}{dy}$$

- The update is called **only on layers which possess weights**.
- You will set the function to call the weight update on the correct layers. The data is stored in **1-D arrays** that hold the tensors. Each tensor is wrapped in a **blob structure** containing **both the data and the gradients**.
- Complete the code of `update_weights()` in `net.c` with the most suitable blobs to update the DNN.
- To answer the question, **print the weights** of the last layer before and after the update, using `print_weights()`.
- Afterwards, **comment out** all the `printf()` and `print_weights()` calls around `update_weights()`.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

04. Assignments – Task 5 – Training the Classifier

- Now, we will run the **full training loop** and test the convergence of the DNN against the sample input and label generated by the GM.
- To start, **uncomment the code** at the bottom of `net.c` (both the **profiling** and the **last block of** `net_step()`). **Remember to comment out all the print functions.**
- The dataset is a **single element**, so training iterates on this one sample for a given number of **epochs** to test convergence and **validate the model**. It is normal to **overfit** on such a demo model.
- Define the `learning_rate` and `epochs` hyperparameters in `GM.py`.
- Leave `batch_size` at 1, as we have a single sample. `batch_size` = 1 is a common setup for training DNNs on microcontrollers, due to their **limited memory**.

```
make clean get_golden all run
```

- This command makes the GM **set up the C code** to run training with the given hyperparameters.
- Compile and run the code with the **2 setups from the assignment document**.

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



Călin Diaconu – calin.diaconu2@unibo.it

Ahmad Mirsalari – seyedahmad.mirsalar2@unibo.it

Ihor Sachok – ihor.sachok@unibo.it